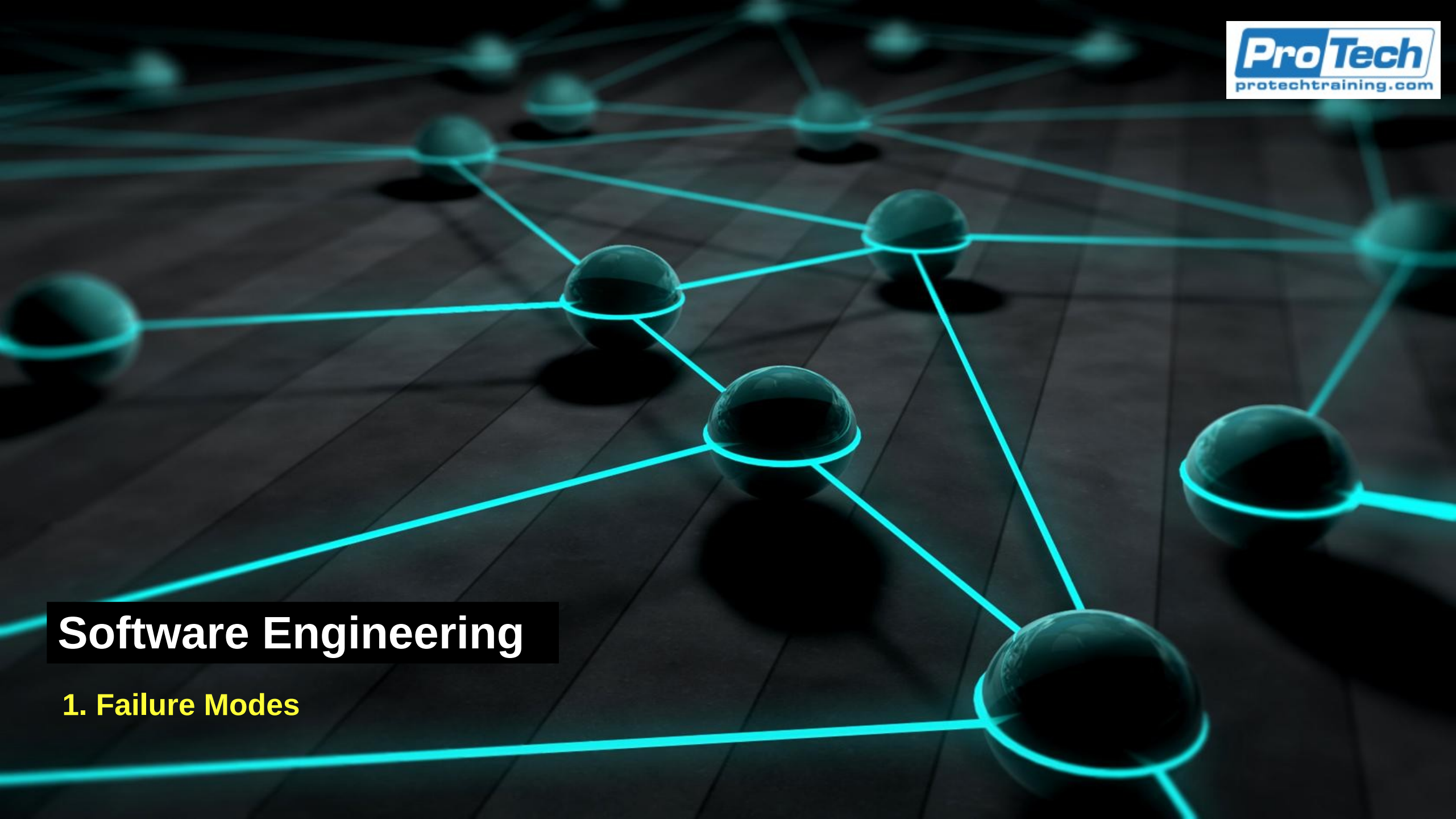


Software Engineering

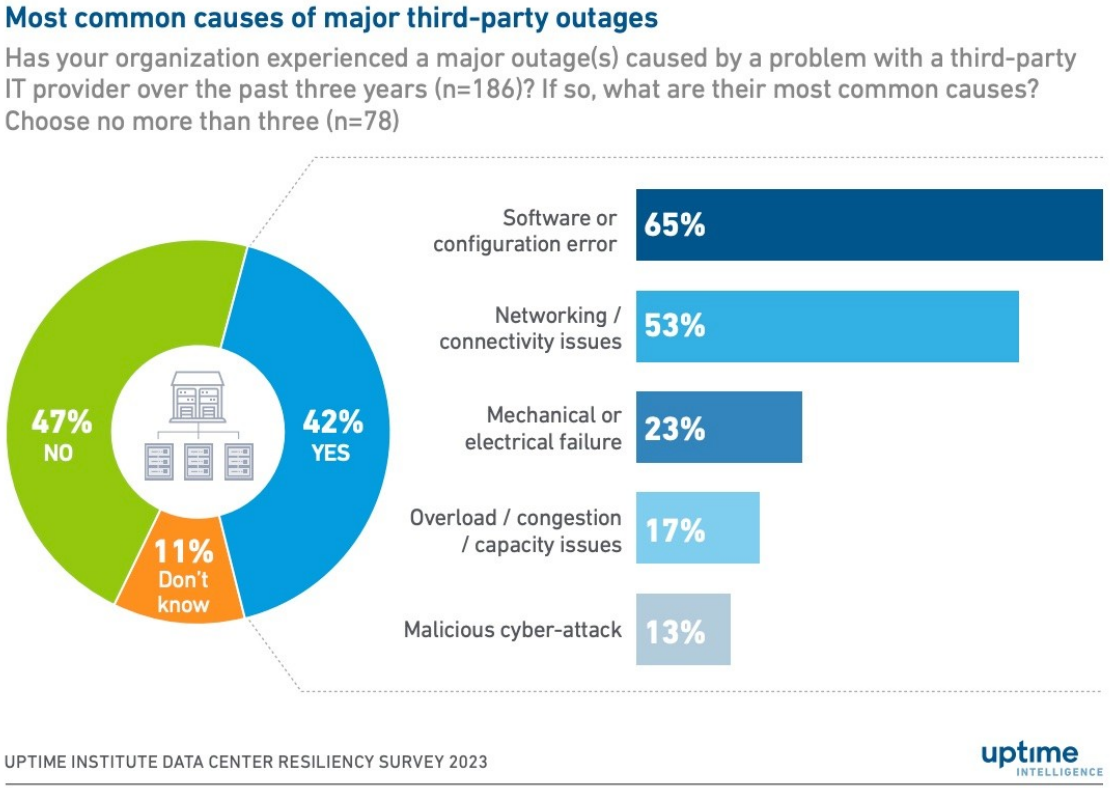
1. Failure Modes

What Failure Really Means in Operations

- This section gives you the precise vocabulary for talking about failure that the rest of the module depends on.
  - It separates several ideas that everyday language jams together under the single word failure.
  - It establishes that failure is usually partial and often invisible, not a clean on or off event.
- It sets up the failure taxonomy that follows, because you cannot name a failure you have no words for.

Failure Is the Normal Background Condition

- Complex systems are essentially always running with some faults present, even when they look completely healthy.
  - The work of operations is to keep a system delivering its service despite the ever-present flaws.
  - This means failure is not a rare event, but a constant pressure that is constantly being mitigated.
- Treating failure as normal, rather than as an unexpected exception, is the core attitude shift from dev to ops thinking



Failure Is the Normal Background Condition

- Complex systems are always running in a partially degraded state, with multiple small faults present at any given moment
  - They keep working not because they are flawless but in spite of those flaws.
- The operator's job is not to achieve a magical state of zero faults
  - But to keep the system delivering its service while faults come and go.
- A failure is not necessarily evidence that someone was negligent or that the system was badly built
  - It is the expected behavior of a complex system under pressure.
  - Engineers who expect perfection are surprised and unprepared when things break
  - Engineers who expect failure design for it, watch for it, and respond to it calmly.

Why We Need Precise Words for Failure

- Everyday language collapses several different ideas into the one overloaded word failure.
- An engineer who says the system failed has not yet said anything precise enough to act on.
- Detection, communication, and diagnosis all break down when the team lacks shared, exact vocabulary.
- The next few slides introduce a small set of careful terms that will be used consistently

Fault, Error, and Failure

- A fault is a flaw or defect in the system
  - Such as a bug, a misconfiguration, or an undersized resource.
- An error is the moment that a fault becomes active
  - When an error occurs, the system starts doing something wrong internally.
- A failure is when the system no longer delivers its expected service to its users.
- A fault can exist for a long time without causing an error
- An error does not always reach the user as a failure.

Fault, Error, and Failure

- A fault is a flaw sitting in the system
  - For example, a bug in the code, a wrong value in a configuration file, a resource that is too small for the load it will eventually see
  - A fault is static because it is present whether or not anything is currently going wrong.
- An error is what happens when that fault becomes active
  - For example, when the flawed code path actually executes or the undersized resource actually runs out.
  - The error is the fault in motion because the system now behaving incorrectly on the inside.
- A failure is when that internal misbehavior reaches the outside world and the system stops delivering its service as promised.
- These do not automatically chain together.
  - A fault can lurk for months without ever being triggered.
  - An error can occur internally and be caught, retried, or absorbed so that no user ever sees a failure.
  - The chain from fault to error to failure can be broken at each step by corrective and preventative controls
-
-

A Worked Example - One Fault, Then a Failure

- Fault
  - A checkout service is configured with a database connection pool that is too small for peak traffic.
  - At low traffic nothing goes wrong, so the flaw sits there quietly and no one notices it.
- Error
  - During a holiday traffic spike every connection is in use, and new requests can no longer get one.
- Failure
  - Requests time out, orders stop completing, and to customers the checkout has clearly failed.

Latent Versus Active Faults

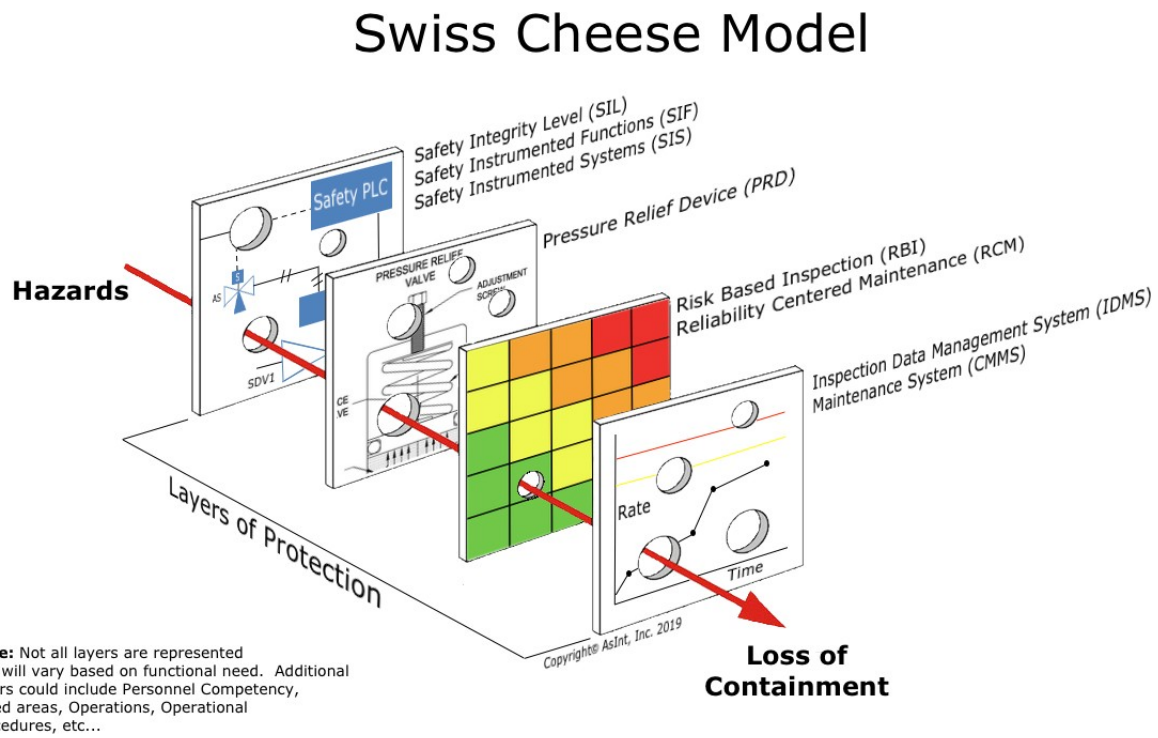
- A latent fault is a flaw that is present in the system but is not currently causing any problem.
  - Latent faults can sit dormant for a long time, which is exactly why they are dangerous.
- A fault becomes active when some condition or trigger finally exercises it.
  - Many serious incidents are simply a latent fault meeting the one condition that wakes it up.
- Latent faults are dangerous is precisely because they are invisible during normal operation.
  - The system looks healthy, all the dashboards are green, and the flaw is sitting there waiting.

Latent Versus Active Faults

- Faults become active when a trigger finally exercises them
  - Nothing was visibly wrong, then one condition arrived that activated a fault that had been present all along.
  - This is why experienced operators are uneasy about the idea that quiet means safe.
  - Quiet often means that the faults present in the system simply have not been triggered yet.
- It is a high priority task to reduce the number of latent faults
  - Relies on good design
  - And relies on catching them through testing and observability before a real trigger finds them first.

The Swiss Cheese Model of Layered Defenses

- Picture each defensive layer in a system as a slice of Swiss cheese, where the holes are weaknesses.
  - No single layer is perfect, so every slice has holes, meaning gaps where a fault can slip through.
  - A failure reaches users only when the holes in many layers happen to line up at the same moment.
- This is why serious failures usually require several things to go wrong together, not just one.



Note: Not all layers are represented and will vary based on functional need. Additional layers could include Personnel Competency, Diked areas, Operations, Operational Procedures, etc...

Partial Versus Total Failure

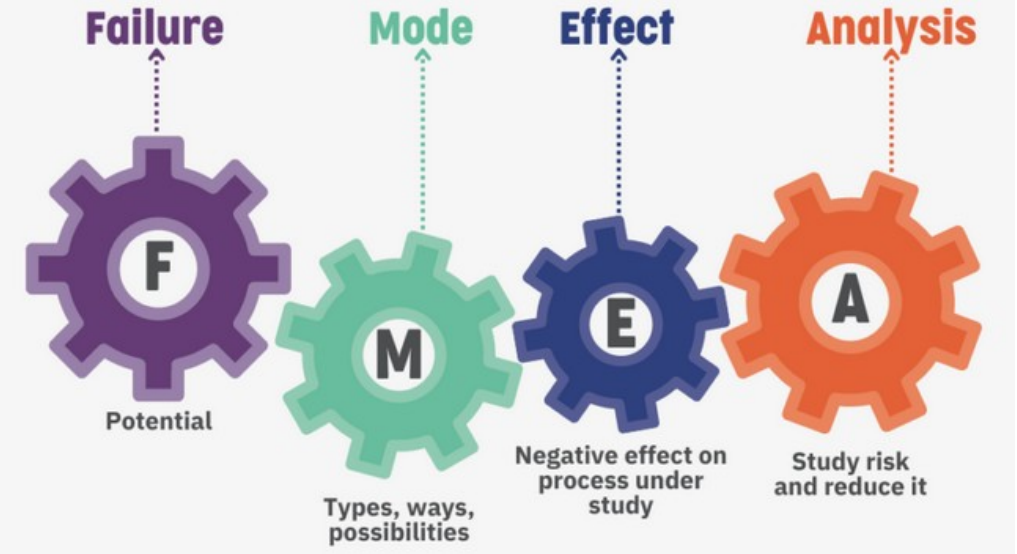
- A total failure is the rare case where the whole system is down for everyone at once.
- A partial failure affects only some users, some requests, some regions, or some features.
- Most real failures are partial, which makes them harder to notice and harder to reason about.
  - In a real distributed system, a failure typically touches only a slice of things.
  - They are harder to notice, because most of the system looks fine
  - They are harder to reproduce, because they depend on specific conditions
  - And they are harder to reason about, because the answer to “Is it working?” is not a clean yes or no but a more vague “sorta”

Gray Failure - When the System Thinks It Is Healthy

- A gray failure is the dangerous case where the system believes it is healthy while users experience it as broken.
  - It happens when the system's own health checks see something different from what real users see.
  - A service can pass every internal check and still be failing the people who actually depend on it.
  - Gray failures are especially harmful because the system does not raise the alarm on itself.
- In a gray failure
  - The system does not alarm on itself, because by its own measures nothing is wrong.
  - So these failures can persist for a long time, with the team unaware, until users complain loudly enough to get noticed.
  - Operations must measure the system the way users actually experience it, not just whether an internal heartbeat is ticking or whether internal metrics are being met
  - One possibility is that we are not observing the right metrics and not getting an accurate picture of the system’s health.

The Failure-Mode Taxonomy

- Part 1 - Resource and performance failures
- This section begins the catalog of specific ways that systems fail, using the vocabulary from the previous section.
- It covers the first family of failures, the ones driven by resources and performance.
- These are failures of running out, slowing down, and getting stuck, rather than failures that feed on themselves.



Running Out of Resources

- Every running system depends on a finite supply of resources to do its work.
  - When demand for a resource exceeds its supply, the system can no longer serve work normally.
  - This whole family of failures comes down to a simple cause: something ran out.
  - Operational engineering identifies which resources can run out
- When the demand for a particular resource grows past the amount available
  - The system can no longer keep up, and work starts to back up, slow down, or fail outright.
  - Referred to as resource saturation
  - There are many that can be exhausted
  - The less obvious resource exhaustion cause some of the most confusing incidents precisely because nobody was watching them.

Saturation and Resource Exhaustion

- Saturation is the state where a resource is being used as fully as it can be, with no spare capacity left.
- A saturated resource forces new work to wait, to be rejected, or to fail.
- Saturation often shows up first as rising latency, before anything actually errors out.
- Watching how close resources are to saturation is one of the most useful early-warning signals in operations.

Saturation and Resource Exhaustion

- Saturation usually announces itself through slowness before it announces itself through errors.
  - As a resource approaches full, work starts to wait for it, and waiting shows up as rising latency.
  - Only when things get truly bad do you start to see outright errors and rejections.
- This ordering, latency first and errors later
  - Latency is a valuable early signal
  - Measuring how close key resources are to saturation, rather than waiting for them to fail, is one of the highest-value things an operator can watch.
    - A resource at sixty percent is comfortable
    - A resource at ninety five percent is a warning
    - A resource at one hundred percent is already hurting users.

Resources That Can Saturate

- The obvious ones are compute resources: CPU time and memory, which beginners already watch.
- Storage resources also saturate, including disk space and the often-forgotten supply of inodes in a file system.
- Concurrency resources are a frequent culprit
  - Connection pools, thread pools, and file descriptors.
- The network itself is a resource
  - Bandwidth and the number of available sockets can run out too.

Resources That Can Saturate

- Compute resources
  - CPU runs out when there is more work than cycles
  - Memory runs out when the system tries to hold more than fits
    - often leading to the operating system killing processes or to heavy swapping that destroys performance.
- Storage resources
  - Disk space is obvious but inodes are the often the forgotten saturated resource
    - A disk can report plenty of free space and still refuse to create new files because it has run out of inodes
- Concurrency resources
  - Extremely common causes of real outages.
    - Connection pools to databases, thread pools that process requests, and file descriptors that the operating system limits per process can all be exhausted while CPU and memory look perfectly healthy.
- Network resources
  - Bandwidth availability drops dramatically under too much load
  - The number of available sockets or ephemeral ports can run out under high connection churn.

Contention, Locks, and Deadlock

- Sometimes work stops moving even though there are plenty of resources available.
- Contention happens when many tasks compete for the same shared thing and end up waiting on each other.
- A lock protects shared data by letting only one task use it at a time, which can create a waiting line.
- Deadlock is the worst case, where tasks each hold something the others need and nothing can ever proceed.

Contention, Locks, and Deadlock

- In saturation, work stops because a resource is used up.
- In contention, work stops even though resources appear available, because tasks are getting in each other's way over something shared.
- When many tasks all need the same shared thing at the same time
  - They cannot all have it at once, so they wait, and that waiting is contention.
- Locks are the most common mechanism behind this.
  - A lock is a way of protecting shared data by ensuring that only one task touches it at a time
    - This is necessary for correctness but creates a waiting line when many tasks want it. If that line gets long
    - Throughput collapses even though the CPU might be nearly idle, which is a confusing signature for beginners.
  - The extreme case is deadlock, where two or more tasks each hold something the others are waiting for, so none of them can ever move forward.
    - Deadlock is a permanent stall rather than a slow one.

Latency as a Failure Mode

- Slow Is the new down
- Latency is the time it takes for the system to respond to a request.
- A service that is too slow to be usable has effectively failed, even though it is technically still responding.
- Users do not care whether a frozen page is erroring or just slow, because the experience is the same.
- Treating excessive latency as a real failure, not a minor performance issue, is an operational mindset shift.

Latency as a Failure Mode

- Slowness is a form of failure rather than a separate, lesser category of problem
  - Latency is simply how long the system takes to respond.
  - A service that responds, but responds so slowly that no one can actually use it, has failed in every way that matters to a user
    - Even though by a naive technical measure it is still up and returning results.
  - From the user's POV, a page that hangs for thirty seconds is indistinguishable from a page that returns an error
    - In some ways it is worse, because the user waits and hopes before giving up.
  - This connects directly back to the gray failure idea from the previous section.
  - A service can be passing its health checks, returning correct responses, and still be failing its users purely because it is too slow.

Tail Latency - Why Averages Hide the Pain

- The average response time can look perfectly healthy while some users have a terrible experience.
- Tail latency is the slow end of the distribution, the requests that take far longer than typical.
- A small percentage of very slow requests can still be a large number of unhappy users at scale.
- This is why operators watch high percentiles, such as the 95th and 99th, rather than the average.

Tail Latency - Why Averages Hide the Pain

- The instinct of a newcomer is to summarize response time with an average which is dangerously reassuring.
- Suppose for a service, almost every request is fast but one in a hundred takes ten seconds.
  - The average might still look fine, comfortably within target
  - But that one in a hundred is a real and miserable experience for a real person.
  - Now scale it up, because scale is what makes the tail matter.
  - One percent sounds small until the service handles millions of requests, at which point one percent is a huge number of frustrated users, and often your most active users
  - Since the more requests someone makes, the more likely they are to hit a slow requests in the tail

Queueing and Backpressure

- A queue is a waiting line for work
  - It acts as a shock absorber for short bursts of load.
- Queues are helpful up to a point
  - They smooth out spikes so the system does not have to reject work.
- Past that point a queue becomes harmful
  - Can result in work waiting so long that it is stale or already abandoned.
- Backpressure is the deliberate signal a system sends upstream to say
  - “Slow down, I cannot take more right now.”

Queueing and Backpressure

- Queues are a double-edged tool, which is a subtle and valuable idea.
  - When a brief burst of load arrives, a queue lets the system accept all of it and work through it steadily, rather than rejecting the overflow.
  - For short spikes, this is exactly what you want.
- The danger is that queues hide a problem while making it worse.
  - If load stays high for longer than a moment, the queue keeps growing, and now work is sitting in line for a long time before it gets handled.
  - By the time a request is finally processed, it may be stale, or the user may have already given up and left, so the work is wasted effort that still consumed resources.
  - A growing queue is therefore one of the clearest warning signs that a system is heading toward trouble.
- Backpressure is a system deliberately signaling upstream that it cannot accept more work right now, asking the sender to slow down or stop.
  - A system with good backpressure protects itself by refusing excess work cleanly, rather than accepting everything and collapsing under an ever-growing queue. .

Utilization and Latency Are Not Linear

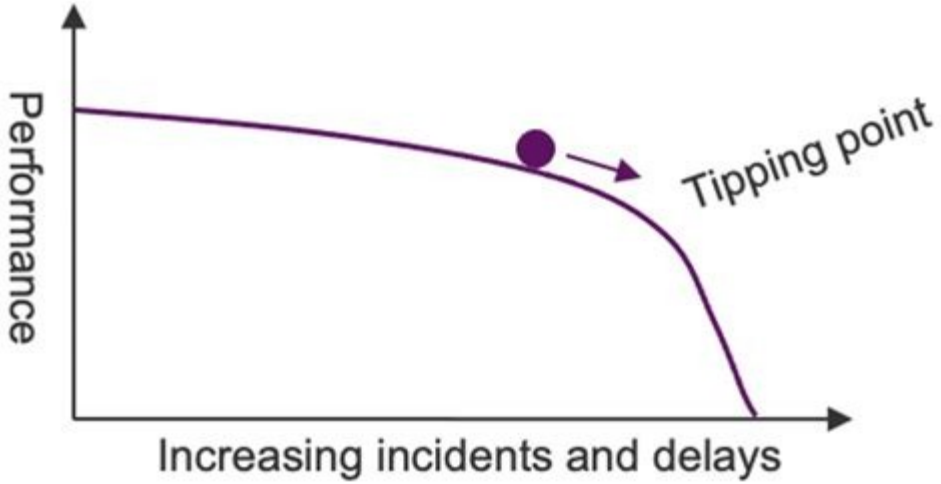
- It is tempting to assume that a system half as busy is half as fast, but that is not how it works.
- As a resource gets close to fully utilized, waiting times do not rise gently, they shoot upward.
- A system at seventy percent can feel fine while the same system at ninety five percent feels broken.
- This is why operators leave headroom on purpose, since the last bit of capacity is extraordinarily expensive.

Utilization and Latency Are Not Linear

- The naive assumption is that load and slowness are proportional
  - Meaning that if you double how busy a system is, you double the response time.
  - The reality, which comes from queueing theory is dramatically different.
  - As a resource approaches full utilization, waiting time does not rise in a straight line.
    - It curves upward sharply and exponentially.
    - Use a highway analogy. A motorway at forty percent capacity flows freely, and at seventy percent it is still moving well.
    - But somewhere around ninety percent, a tiny additional increase in cars produces a massive increase in delay, and the traffic jam appears suddenly.
    - The road did not get a little worse, it tipped from flowing to jammed almost all at once.
  - Software systems behave the same way.
    - A service at seventy percent utilization can feel perfectly responsive
    - The very same service at ninety five percent feels broken, because the waiting time has exploded.
  - Good operators deliberately leave headroom and do not run systems near full capacity, because that last sliver of utilization is purchased at an enormous and nonlinear cost in latency.
- Running hot is not efficient, it is fragile.

The Knee - Where Coping Tips Into Collapse

- Saturation and latency are really two views of the same thing, a system approaching its limits.
- There is a point, often called the knee, where a system stops degrading gently and starts collapsing.
- Below the knee, more load means a little more latency, and the system absorbs it gracefully.
- Above the knee, more load means runaway latency and failure, which is the tipping-point idea from Module 1.
-



Utilization and Latency Are Not Linear

- Resource failures and the latency failures we have discussed are not separate phenomena.
- They are two ways of describing the same underlying reality
  - a system being pushed toward its limits.
  - Saturation is the resource view, the tank running dry
  - Rising latency is the experiential view, the work taking longer and longer to get done.
  - They climb together.
    - If you picture load on one axis and response time on the other, the curve stays low and gentle for a long time and then bends sharply downward at a particular point.
    - That bend is the knee where the system is collapsing.
    - Adding even a little load now produces runaway latency and outright failure.
    - This is the the nonlinearity and thresholds idea we introduced in earlier
  - Systems do not usually fail by sliding smoothly from healthy to broken.
  - They sit in a comfortable zone and then tip over a cliff

Retry Storms and Retry Amplification

- When a request fails, the natural response is to retry it, which works well for a brief transient glitch.
- But when the system is already overloaded, every retry adds even more load to the thing that is struggling.
- Retries also multiply, because one original request can become three or more attempts under the hood.
- The retry, meant to help the user, instead deepens the overload and helps keep the system down
- Often referred to as “The Thundering Herd” of retry attempts

Retry Storms and Retry Amplification

- Retries are not a mistake, they are usually correct.
  - If a request fails because of a brief, isolated glitch, retrying it is exactly the right thing to do, and it quietly fixes a huge number of transient problems without anyone noticing.
- The trouble begins when the failure is not an isolated glitch but a sign that the whole system is overloaded.
  - Now retrying is the worst possible response, because the system is failing precisely because it has too much work, and a retry adds still more work.
  - This is the loop: Overload causes failures, failures cause retries, and retries cause more overload.
  - On top of that, retries amplify.
    - If every client retries a failed request two or three times, then one original request becomes three or four attempts hitting the backend, so the effective load can triple or quadruple at the exact moment the system can least afford it.
  - Connect this back to the knee from the previous section.
    - A system sitting just below its knee can be shoved well past it by a retry storm, and then it stays there.

Thundering Herd and Synchronized Behavior

- A thundering herd is when a large number of clients all do the same thing at the very same moment.
  - This often strikes right after a recovery, when everyone who was waiting rushes back in at once.
- It also happens when clients are accidentally synchronized, such as all retrying on the same fixed timer.
- The sudden synchronized surge can immediately knock over the very system that just came back to life.

Thundering Herd and Synchronized Behavior

- The thundering herd is a close cousin of the retry storm, and it has a particularly cruel signature.
- The core problem is synchronization.
  - When many clients act in unison, their combined load arrives as one giant spike rather than a smooth stream
  - That spike can be far more than the system can handle even though the same clients spread out over time would be fine.
  - The classic version happens right after a recovery.
    - A service goes down, thousands of clients are waiting and retrying, and the instant the service comes back
    - All of them hit it at the same moment.
    - The service, which is fragile and cold immediately after recovering, gets flattened again by the very clients that were waiting for it.
  - Synchronization creeps in through innocent-looking choices.
    - Clients that all retry on a fixed one second timer are synchronized.
    - Scheduled jobs that all run exactly on the hour are synchronized.
    - Cached items that were all created at the same time expire at the same time.

Cache Stampede and Cold-Cache Effects

- A cache stores recent results so the backend does not have to recompute or refetch them every time.
- The cache quietly absorbs most of the traffic, so the backend usually sees only a small fraction of it.
- A cache stampede happens when many cached entries expire together and the full load slams the backend at once.
- A cold cache, such as just after a restart, causes the same thing, since nothing is cached to absorb the load yet.

Cache Stampede and Cold-Cache Effects

- This is a hidden fragility that catches many teams by surprise.
  - A cache sits in front of an expensive backend and serves repeated requests from memory, so the backend does not have to do the same expensive work over and over.
  - The important and easily forgotten consequence is that the backend may only ever see a small slice of the real traffic, perhaps five percent, because the cache absorbs the rest.
  - Over time, people forget this, and the backend ends up sized for the small slice it normally sees, not for the full firehose of requests
- Now consider what happens when the cache suddenly stops absorbing load.
  - In a cache stampede, many entries that were created at the same time expire at the same time
    - All the requests that were being served from cache now miss simultaneously and rush to the backend at once.
    - In a cold-cache situation, such as right after a restart or a cache flush, there is nothing in the cache at all, so every request goes straight through.
    - In both cases, a backend that was comfortably handling five percent of the traffic is abruptly hit with one hundred percent, and it saturates instantly.
    - Notice that this is both a thundering herd, because the misses are synchronized, and a feeding loop, because the now-overloaded backend gets slower, which causes more requests to pile up and miss. Tell the learners that the cold cache after a deployment or restart is one of the most common real-world incidents they will encounter.

Metastable Failure - the Trap That Sustains Itself

- A metastable failure is one that keeps going even after the original trigger has completely gone away.
- The system was pushed past its tipping point, and now a feedback loop sustains the failure on its own.
- Removing the original cause does not fix it, which is both deeply counterintuitive and very dangerous.
- The system effectively has two stable states, a healthy one and a failed one, and it is stuck in the failed one.

Metastable Failure - the Trap That Sustains Itself

- A metastable failure is one where you can remove the thing that caused it and the failure continues anyway.
  - This violates the intuition that fixing the cause fixes the problem, and that violated intuition is exactly what makes these failures so hard to handle.
  - Imagine a brief load spike pushes a service past its knee, so requests start timing out.
  - The clients, seeing timeouts, retry.
    - The retry load alone is enough to keep the service saturated, even after the original spike is long gone.
    - The trigger has vanished, but the loop it started is now self-sustaining. Introduce the language of two stable states.
    - A healthy system sits in a good stable state, low load and fast responses.
    - But there is a second stable state, the failed state, where the system is saturated and the sustaining loop keeps it that way.
  - A metastable failure is what happens when a trigger knocks the system out of the good state and into the bad one, where it now stably sits.

Escaping Metastable Failure - Recovery Is Not Automatic

- Because the failure sustains itself, you usually cannot fix it by waiting or by removing the trigger.
- You have to actively break the feedback loop, and most often that means reducing the load on the system.
- Operators escape these failures by shedding load, draining backlogs, or temporarily turning clients away.
- The trigger and the cure are different things, so chasing what started it will not help you recover from it.

Escaping Metastable Failure - Recovery Is Not Automatic

- Operators shed load by deliberately dropping a fraction of requests
- They drain or discard the backlog of queued work that is feeding the loop
- They temporarily disable client retries, and sometimes they turn traffic away entirely for a short period to let the system recover.
- You may have to reject real user traffic in order to save the system for everyone, because a system stuck in the failed state is serving nobody well.
- Good design, like backpressure and load shedding built in from the start, prevents systems from falling into these states at all.

Cascading Failure - the Domino Effect

- A cascading failure spreads from one failed component to another, one after the other.
- When a component fails, the work it was handling does not vanish, it shifts onto whatever remains.
- The remaining components, now carrying extra load, become more likely to be pushed past their own limits.
- Each new failure dumps its load onto the survivors, so the collapse speeds up like falling dominoes.

Cascading Failure - the Domino Effect

- The failure engine, once again, is a reinforcing loop
  - But here the loop runs across components rather than within one.
- When a component fails, the work it was responsible for does not simply disappear.
  - It gets redirected to whatever components are still standing.
  - Those survivors are now handling their own load plus a share of the failed component's load, which pushes them closer to their own limits, and possibly past their own knee.
  - When one of them falls, its load redistributes again onto an even smaller set of survivors, who are now even more overloaded.
  - Each failure makes the next one more likely, and the pace accelerates, which is why we picture it as falling dominoes.
  - Cascading failure travels along the connections between components, which means you cannot predict or contain it without understanding your dependencies

Correlated Failure - When Redundancy Is an Illusion

- Redundancy means running several copies, so that if one fails, the others keep the service running.
- Redundancy only protects you if the copies fail independently, for reasons unrelated to one another.
- A correlated failure is when many copies fail at the same time because of the same underlying reason.
- Three replicas give you no protection at all if a single shared cause can take all three down at once.

Correlated Failure - When Redundancy Is an Illusion

- Redundancy is the standard answer to reliability.
  - If one copy might fail, run three, and the odds of all three failing at once seem reassuringly tiny.
- But that hides a critical assumption, which is that the copies fail independently, meaning the failure of one tells you nothing about the others.
- A correlated failure is what happens when that assumption is false
  - Several copies now fail together because they share a common cause.
  - Three replicas feel safe, but if all three are running the same software version and a bad deployment goes out, one cause takes all three down simultaneously.
  - If all three sit in the same rack and the rack loses power, one cause takes all three.
  - If all three depend on the same database and that database goes down, again, all three fall together.
- In each case, the redundancy provided no protection against that particular cause, because the cause was shared.

Shared Fate - the Hidden Links Behind Correlated Failure

- Shared fate is when components that look independent are secretly linked by something they have in common.
- Physical shared fate includes the same power supply, the same rack, the same data center, or the same region.
- Logical shared fate includes the same upstream dependency, the same configuration, or the same deployment.
- Real redundancy is the work of finding and removing shared fate, so that copies truly fail for separate reasons.

Shared Fate - the Hidden Links Behind Correlated Failure

- Shared fate
  - Any hidden link that causes things you believed were independent to actually rise and fall together.
  - The job of an operator is to find these links before they bite.
- Two broad categories.
  - Physical shared fate is about location and infrastructure.
    - Copies that share a power supply, a rack, a cooling system, a data center, or a cloud region can all be taken out by a single physical event, no matter how separate they look in the architecture diagram.
  - Logical shared fate is about software and configuration.
    - Copies that share the same upstream dependency, such as a single database, authentication service, or DNS, all fail when that dependency fails. Copies running the same code version all share any bug in it, so one bad deployment hits them all.
    - Copies sharing the same configuration or the same expiring certificate share that single point of failure too.

Why Dependencies Are Where Failure Actually Lives

- Module 1 made the point that system behavior emerges from interactions between parts, not from parts in isolation.
- A dependency is any other component or service that a piece of work relies on in order to do its job.
- Most outages do not happen inside a single component, they happen in the links between components.
- To reason about how a failure spreads, you first have to be able to see the dependencies it travels along.

Why Dependencies Are Where Failure Actually Lives

- A dependency is anything a piece of work relies on to get done
  - Another service it calls, a database it reads, an external API it waits on, a configuration it loads.
- For real outages, surprisingly few of them are a single component failing in isolation.
  - Far more often, the trouble is in the connections, a slow dependency dragging down its caller
  - A failure in one service rippling out to others, a chain where any broken link breaks the whole.
- Cascading and correlated failure both travel through these dependency links.
- If failure lives in the connection
  - Then understanding your system's reliability means understanding its dependency graph.
  - You cannot predict, contain, or diagnose a propagating failure without a map of the dependencies it can move through

Job and Request Flow - Mapping the Path of Work

- A useful first move is to trace the actual path that a single unit of work takes through the system.
- For an online system this is the request flow, the sequence of hops a user request makes from entry to response.
- For batch or background systems this is the job flow, the stages a piece of work moves through over time.
- Mapping the real path, rather than the idealized architecture diagram, is the core skill behind dependency analysis.

Job and Request Flow - Mapping the Path of Work

- For example placing an order, and then literally tracing what happens.
  - The request arrives at a load balancer
  - Goes to a web server
  - Which calls an authentication service
  - Which calls an inventory service
  - Which reads a database
  - Which then calls a payment provider, and so on.
- That traced path is the request flow

Job and Request Flow - Mapping the Path of Work

- For systems that do work in the background rather than in response to a user, the same technique applies but we call it job flow
- The stages a unit of work passes through as it is processed
  - Perhaps pulled from a queue
  - Transformed
  - Written to storage, and acknowledged.
- The real path that work takes is frequently different from the clean architecture diagram on the wall
  - Diagram leaves things out, hides shared infrastructure, and quietly omits steps that everyone forgot about.
  - The skill is to trace the actual path, not the idealized one.

Hard Versus Soft Dependencies

- A hard dependency is one the operation cannot complete without, so if it fails, the operation fails.
- A soft dependency is one the operation can survive without, often by running with reduced functionality.
- The very same downstream service can be a hard dependency for one operation and a soft one for another.
- Classifying dependencies as hard or soft tells you which failures will actually take you down and which will not.

Hard Versus Soft Dependencies

- If you are making a purchase
  - The payment service is a hard dependency, because there is no completing the purchase without it, so when it fails, the operation fails.
  - A soft dependency is one the operation can do without, perhaps in a degraded form.
  - A product recommendations service is usually soft, because if it is down, you can still show the product page, just without the recommendations panel.
- The same service can be hard for one flow and soft for another
  - The classification is not a property of the service alone but of the service in the context of a particular operation.
  - A soft dependency that is accidentally treated as hard is a self-inflicted wound.
    - If your page blocks and waits for the recommendations service before rendering anything, then you have turned an optional feature into a hard dependency
    - And its failure now takes down the whole page for no good reason.
  - A core resilience move, is to deliberately turn hard dependencies into soft ones wherever possible, so that more of the system can keep working when pieces of it fail.

Other Dependency Dimensions Worth Naming

- A synchronous dependency makes the caller wait for a response, tying the caller's fate to it in real time.
- An asynchronous dependency lets the caller hand off the work and continue, which loosens the coupling between them.
- An internal dependency is one your own team controls, while an external one belongs to a third party.
- Synchronous and external dependencies deserve extra caution, since you wait on them most and control them least.

Other Dependency Dimensions Worth Naming

- Two useful axes for classifying dependencies
- The first axis is synchronous versus asynchronous.
  - A synchronous dependency is one where the caller stops and waits for an answer, which means the caller is tightly coupled to it in time.
  - If the dependency is slow, the caller is slow, and if the dependency fails, the caller is stuck waiting.
  - An asynchronous dependency is one where the caller hands off the work, perhaps by putting a message on a queue, and then continues on its way.
  - This loosens the coupling, because the caller is no longer held hostage to the dependency's immediate health.

Other Dependency Dimensions Worth Naming

- Two useful axes for classifying dependencies
- The second axis is internal versus external.
  - An internal dependency is something your own organization owns and can fix.
  - An external dependency belongs to a third party, like a payment processor or a cloud service
    - Which means when it breaks you often cannot do anything except wait
    - And you are also subject to its rate limits and its own outages.
- The riskiest dependency of all is one that is both synchronous and external, because you are forced to wait on something you have no power to fix.

Implicit and Hidden Dependencies

- An explicit dependency is one you already know about and can point to in the architecture.
- An implicit or hidden dependency is one nobody mapped, often discovered only at the moment it fails.
- Common hidden dependencies include DNS, time and clocks, authentication, configuration, and shared infrastructure.
- These produce some of the most surprising outages, precisely because no one was watching them.

Implicit and Hidden Dependencies

- The hidden or implicit dependencies are the ones nobody wrote down, the things the whole system quietly relies on without anyone thinking about them.
- Classic examples
  - DNS is the classic case, so much so that experienced engineers joke that the cause is always DNS, because almost everything depends on name resolution and almost nobody monitors it.
  - Time and clock synchronization is another, since many systems break in strange ways when clocks drift.
  - Authentication and identity services are hidden hard dependencies for nearly every operation.
  - Configuration and secret stores, service discovery, shared databases, and shared network paths all belong on this list too.

The Critical Path

- The critical path is the chain of dependencies that absolutely must all work for a given operation to succeed.
- It is made up of the hard dependencies for that specific operation, linked together from end to end.
- Every component sitting on the critical path is a potential single point of failure for that operation.
- Identifying the critical path tells you where to focus reliability effort and where a failure will hurt the most.

The Critical Path

- The critical path for an operation is the chain of things that all have to work for that operation to succeed
  - It is built precisely from the hard dependencies, strung together in sequence.
  - Everything on the critical path is, by definition, something whose failure breaks the operation
  - Which means every component on it is a potential single point of failure unless it has been made redundant.
  - Once you know the critical path for an important operation, you know exactly where to spend your reliability budget
  - Because those are the components whose failure you cannot tolerate.
  - You also know which components are off the critical path, the soft dependencies whose failure only degrades the experience rather than breaking it.

Fan-Out and Fan-In

- Fan-out is when handling a single request requires making many downstream calls, sometimes dozens or hundreds.
- Fan-in is when many separate requests or results converge onto a single downstream component.
- Fan-out multiplies risk, because the whole request is only as fast and as reliable as its slowest, weakest call.
- Fan-in concentrates load, because one component has to absorb the combined demand of everything pointing at it.

Fan-Out and Fan-In

- Fan-out is when one incoming request explodes into many outgoing calls.
  - A single page load might call dozens of backend services to assemble everything it needs to show.
- The danger of fan-out is twofold
  - First, latency. If the request has to wait for all of its many downstream calls before it can respond, then its speed is governed by the slowest of them.
  - Second, reliability. If any required downstream call fails, the request can fail, so chaining many calls together raises the odds that at least one breaks.
  - Fan-out therefore multiplies both the latency risk and the failure risk.

Fan-Out and Fan-In

- Fan-in is the mirror image
  - Many callers all point at one shared component.
- The danger there is concentration.
  - That single component has to absorb the combined load of everything depending on it, which makes it both a bottleneck and a single point of failure
  - And it is exactly the kind of place where saturation and contention show up.

The Math of Dependency

- Imagine a request that needs ten services, each one independently available 99.9 percent of the time.
- Because their availabilities multiply together, the request as a whole is available only about 99 percent of the time.
- This means that many individually reliable parts can still add up to an unreliable whole.
- The more hard dependencies you chain onto the critical path, the lower the combined availability quietly becomes.

The Math of Dependency

- Suppose an operation depends on ten services
  - Each of them is individually very reliable, available 99.9 percent of the time, which is a number teams are often proud of.
  - The instinct is to assume the whole operation is therefore about 99.9 percent reliable too. That instinct is wrong, and dangerously so.
  - When you have a chain of hard dependencies that all must work, their availabilities multiply.
    - Ninety nine point nine percent multiplied by itself ten times comes out to about ninety nine percent.
  - One service at 99.9 percent is down roughly forty three minutes a month
  - The combined operation at 99 percent is down roughly seven hours a month
  - An order of magnitude worse, purely from chaining reliable parts together.

Failure Domains and Blast Radius

- A failure domain is a boundary within which a failure is contained and beyond which it should not be able to spread.
- Blast radius is the scope of impact when something fails, meaning the full set of things harmed by that failure.
- Good design draws deliberate boundaries, so that a failure inside one domain cannot take down the others.
- The goal is to make the blast radius of any single failure small and predictable, rather than letting it go system-wide.

Failure Domains and Blast Radius

- A failure domain is a containment boundary
  - A region of the system designed so that a failure inside it stays inside it.
- Blast radius is the measure of how far a failure actually reaches
  - The complete set of users, operations, and components affected when something breaks.
- A cascading failure is exactly what happens when there are no effective boundaries
  - Because the dominoes can fall right across the whole system, giving a system-wide blast radius.
  - Failure domains are the walls that stop the dominoes.
- For example
  - Running copies in separate availability zones so a zone outage is contained
  - Splitting users or data into independent shards or cells so a problem in one does not touch the others
  - Separating critical operations from non-critical ones so a failure in a minor feature cannot reach a core flow.

The Types of Change That Ignite Incidents, Part 1

- A code deploy is the most obvious change, shipping new or modified software into production.
- A configuration change alters how the system behaves without changing the code, and it is a frequently underestimated culprit.
- A feature-flag flip turns functionality on or off at runtime, which is still a change even though nothing was deployed.
- These changes are deliberate and human-initiated, which is why a careful deployment process matters so much.

The Types of Change That Ignite Incidents, Part 1

- The code deploy is where new or modified software goes into production.
  - Everyone accepts that a deploy is a change and a risk.
- Spend real time on configuration changes, because they are dangerously underestimated.
  - People often treat a config change as not a real change, something minor that does not need the same review or testing as code
  - That attitude causes a startling number of major outages.
  - A single wrong configuration value can be pushed instantly and everywhere, with no compile step and sometimes no review, and take down a global service in seconds.
- Feature-flag flips are subtle.
  - Modern teams deliberately separate deploying code from releasing functionality by hiding new features behind flags, then turning them on later.
  - The trap is that flipping a flag does not feel like a deploy, because no artifact ships
  - But it absolutely changes the behavior of the production system and can absolutely cause an incident.

The Types of Change That Ignite Incidents, Part 2

- An infrastructure change alters the platform underneath the application, such as networking, scaling, or the operating system.
- A dependency or version update brings in new behavior from a library, service, or runtime that you did not write.
- A shift in data or traffic is a change too, even with no deploy, such as a new usage pattern or a sudden surge.
- These changes are easy to overlook, because some of them are not initiated by your team at all.

The Types of Change That Ignite Incidents, Part 2

- Some changes are easy to forget precisely because they do not look like a normal deploy
- Infrastructure changes, the first type, sit underneath the application, in the networking, the scaling rules, the operating system, the cloud platform, or the orchestration layer.
  - A change in that layer can break the application above it even though the application code never moved.
- Dependency and version updates, the second type, represent someone else's change becoming your problem.
  - When a library you use ships a new version with different behavior, or a downstream service you call gets updated
  - You have inherited a change you did not make and may not even know about.
- The third type is a shift in data or traffic.
  - If your users suddenly send a new shape of input, or a new client starts querying you in a different pattern
  - The system is now operating under conditions it was not operating under yesterday, and that is a change even though nobody deployed anything.

Failure Without a Deploy

- Sometimes a system that nobody touched fails anyway, which feels mysterious but has a clear explanation.
- A latent fault that was present all along can be activated by a change in the environment rather than by a deploy.
- The environment around a system changes on its own through growth, the passage of time, and accumulating state.
- So we did not change anything is rarely the end of an investigation, because the world around the system changed.

Failure Without a Deploy

- A latent fault is a flaw that has been sitting in the system harmlessly, waiting for a trigger.
  - The resolution to the mystery is that the trigger does not have to be a deploy.
  - The environment around the system is never truly static, even when the code is frozen.
  - It changes on its own in at least three ways. It grows, as load and data volume slowly increase until something crosses a threshold
  - Time passes, and some things are sensitive to time. And state accumulates, as logs pile up, caches grow, and tables get larger.
  - Any of these can be the change that finally activates a latent fault, with no deploy involved at all.
  - Experienced operators are skeptical of the phrase “nothing changed.”
  - It is almost never literally true, because even if your code did not change, the world your code runs in did.

Time Bombs - Failures That Arrive on a Schedule

- Some latent faults are time bombs, primed to go off at a particular moment regardless of any deploy.
- Expiring TLS certificates are the classic example, working perfectly right up until the instant they expire.
- Disks that slowly fill, identifiers that run out, and counters or dates that roll over are all time bombs.
- These failures are predictable in principle, which is exactly why monitoring for them pays off so well.

Time Bombs - Failures That Arrive on a Schedule

- The defining feature is that the failure is scheduled
  - Even though nobody scheduled it on purpose.
  - The classic and painful example, is the expiring TLS certificate.
  - A certificate works flawlessly, securing traffic without a hiccup, right up until the precise moment it expires
  - At which point connections start failing everywhere that relied on it.
  - It is completely predictable, since the expiry date is known in advance
  - Yet certificate expiry remains one of the most common causes of self-inflicted outages, because nobody was watching the calendar.

The Anatomy of an Incident

- Most real incidents share a common four-part structure that ties this entire module together.
- First there is a latent fault, a flaw that has been sitting quietly in the system, waiting.
- Then a trigger, very often a change, activates that fault and starts something going wrong.
- The problem propagates through dependencies, and an amplifying feedback loop turns it into a full outage.

The Anatomy of an Incident

- The first element is a latent fault, the flaw sitting in the system, which we defined in the opening section.
- he second is a trigger that activates it, which this section has shown is usually some form of change.
- The third is propagation, where the now-active problem spreads through the system along the dependency paths we mapped in the previous section.
- The fourth is amplification, where a reinforcing feedback loop, the kind we cataloged in the self-feeding failures section, takes a contained problem and blows it up into a full outage.
- A real outage is usually not one thing breaking
  - It is a chain of several things lining up, each one drawn from a different part of this module.

A Worked Example - One Incident, All Four Elements

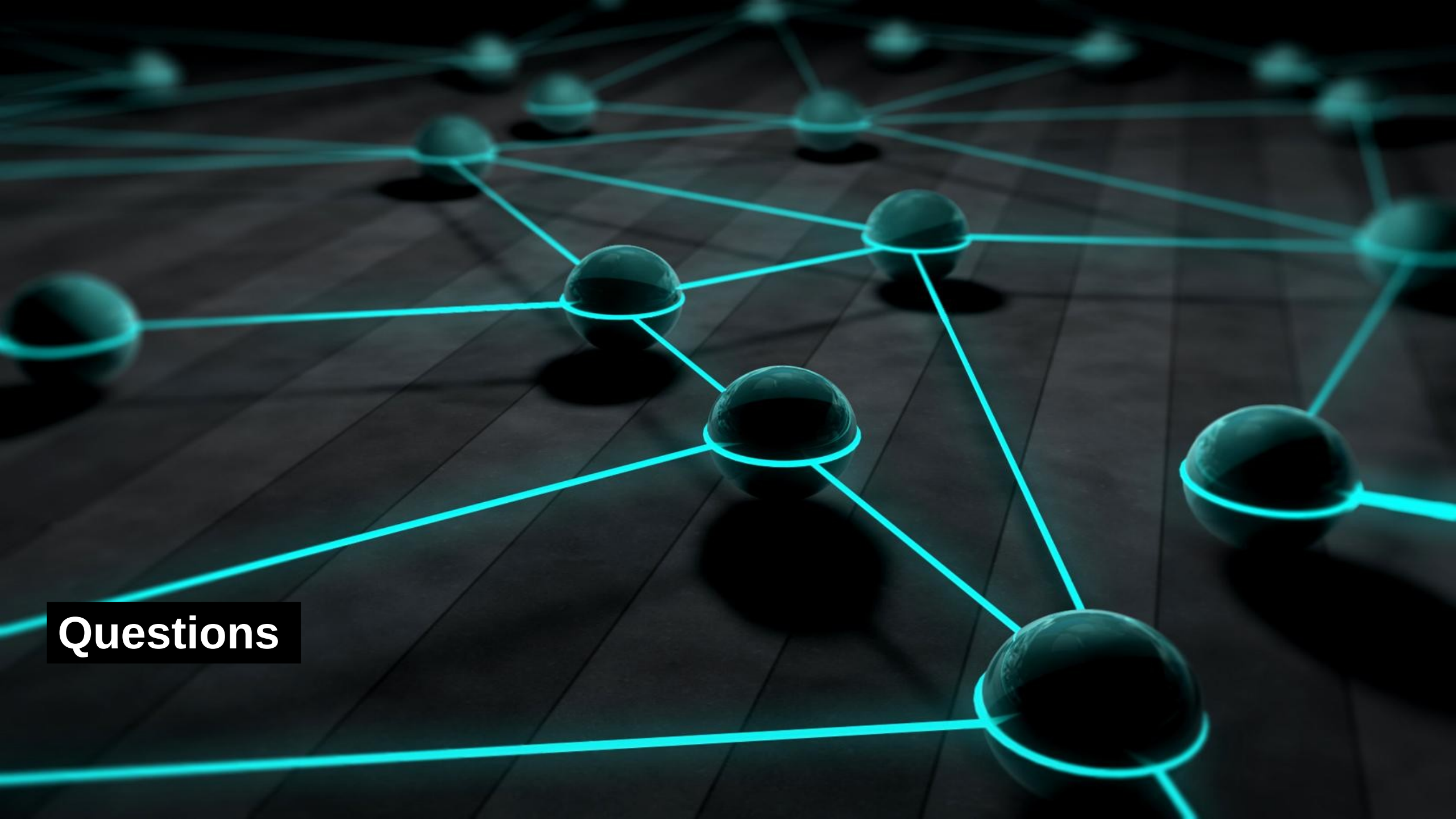
- Latent fault: a service has a connection pool too small for a real peak, sitting harmless for months.
- Trigger: a configuration change lowers a cache timeout, quietly increasing the load that reaches that service.
- Propagation: the service saturates, slows down, and its callers on the critical path begin to time out.
- Amplifying loop: the timeouts cause retries, the retries deepen the saturation, and the system tips into a metastable outage.

Why "What Changed?" Is the First Question

- When something breaks, the most productive opening question is almost always what changed recently.
- Because change is the leading trigger, the recent change log is the highest-yield place to begin looking.
- This instinct shapes both how we watch systems in real time and how we investigate them afterward.
- It points directly to the next two modules, detecting failure as it happens and diagnosing it after the fact.

Why "What Changed?" Is the First Question

- We began by defining failure precisely, separating fault, error, and failure, and recognizing partial and gray failures.
- We cataloged how systems fail, through resource and performance limits and through self-reinforcing feedback loops.
- We mapped the structure that failure travels through, the dependencies, the critical path, and the blast radius.
- And we identified change as the usual spark, with the anatomy of an incident tying every piece together.


Questions